



Security Assessment

Final Report



ReytsCAD

May 2026

Prepared for Reyts

Table of Contents

Project Summary.....	3
Project Scope.....	3
Project Overview.....	3
Protocol Overview.....	3
Assessment Methodology.....	5
Threat and Security Overview.....	6
Findings Summary.....	11
Severity Matrix.....	11
Detailed Findings.....	12
High Severity Issues.....	14
H-01: Due to storing all refHashes in the instance storage of the old mint controller, the new mint controller will be permanently DoSd after ~1300-1400 mints.....	14
Medium Severity Issues.....	16
M-01: Rate band override mechanism can be circumvented with a sequence of transactions.....	16
M-02: Deposit cap checks are ineffective as they're per partner, rather than per asset.....	19
Low Severity Issues.....	21
L-01: Not removing certain instance key-value pairs but only setting them to false can lead to storage bloat.....	21
L-02: vault.draw does not check partner_allowed.....	23
L-03: Governance has no per tx/ daily cap on treasury_send and burn_from_pool.....	24
L-04: All propose_* functions silently overwrite pending state.....	25
L-05: The treasury address in the Governance contract can't be changed.....	26
Informational Severity Issues.....	28
I-01: Use __constructor instead of initialize.....	28
I-02: WASM upgrade inherits SAC allowance.....	29
I-03: First update_rate on a pair leaves PrevRate = 0.....	30
I-04: No check to make sure that throttled_max is less than max_draw_per_tx.....	31
I-05: If the contract isn't initialized at 00:00 GMT+0 (Stellar timestamp), the daily caps/limits for the first day will be valid for less than 24 hours.....	32
I-06: The new minting_gateway daily cap must remain less than the old_minting_gateway daily cap... 33	
I-07: draw does not validate payment_amount against the on-chain rate.....	34
General Recommendations.....	35
Findings Conditional on Admin Integrity.....	35
Monitoring Candidates.....	36
Disclaimer.....	38
About Certora.....	38

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
ReytsCAD	https://github.com/reytsfi/ntech/ReytsCAD/	97b2c84 (original commit) 502d09a (fix commit)	Stellar

Project Overview

This document describes the security review of **ReytsCAD**. The work was undertaken from **May 11th, 2026** to **May 22nd, 2026**.

The following contract list is included in our scope:

contracts/governance/src/lib.rs
contracts/vault/src/lib.rs
contracts/mint-controller/src/lib.rs

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The codebase implements ReytsCAD: a CAD-pegged digital settlement asset on Stellar, designed for use only inside Reyts (a peer-to-peer fiat exchange) and a small set of approved partners. Three Soroban smart contracts gatekeep an off-chain workflow that handles PSP confirmation, reconciliation, and KMS-backed signing. The three contracts in scope:

- **Mint Controller** ([contracts/mint-controller/src/lib.rs](#)) — the sole on-chain path through which new ReytsCAD supply is created. Chains every mint through a legacy Mint Controller that currently holds SAC admin authority. Enforces a daily cap, replay protection, pause, and 24h timelocked upgrade.

- **Governance** ([contracts/governance/src/lib.rs](#)) — moves existing ReytsCAD from Treasury to allowlisted hot wallets and burns from a Burn Pool. Holds no funds itself; operates as a spender against SAC allowances granted by Treasury and Burn Pool. 24h WASM upgrade timelock and 5h SAC admin handoff timelock.
- **Vault** ([contracts/vault/src/lib.rs](#)) — holds partner-deposited NGNC, lets OPS draw against it for cross-currency settlements, and gives partners an unconditional emergency-exit path. Per-partner deposit caps, daily velocity, rate-band throttle, and a 3-tier role model (admin / OPS / partner).

The ReytsCAD token itself is a Stellar Asset Contract (SAC) at a fixed mainnet address. The SAC bytecode is the canonical Stellar template and is out of scope.

Off-chain components — the NestJS API, reconciliation engine, KMS signers, and STOP MODE guard — are out of audit scope but inform the threat model below: several risks in the contracts are bounded only by these off-chain controls.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

Threat analysis for the three Soroban contracts in scope. Each section identifies the affected components, the risk, and the properties verified during review.

1. Mint Authority & Chain-Through Integrity

Affected Components: `mint-controller/src/lib.rs` (`mint_to_treasury`, `initialize`, `set_treasury`), legacy old MC at `CDU4RZ7QEWWTYBIJEJA2SXKIPVG4MZY55O5KNWAHQLRKADYXDWDWIHA4`.

Risk: New MC enforces all validation and replay protection, but the actual `SAC.mint` call happens inside the immutable old MC. New MC discards the old MC's return value; a silent old MC failure or ABI drift consumes daily-cap budget and burns a replay slot without producing real supply.

What we are verifying:

- OPS auth, pause check, `amount > 0`, replay check, and daily-cap check all run before the chained call.
- `mark_ref_used` and `set_minted_today` write state before invoking the old MC (CEI-correct).
- Daily-cap arithmetic uses `checked_add`.
- The chain-through assumption (old MC's stored `Ops` equals new MC's contract address) is not enforced on-chain and depends on old MC's admin discipline.
- No on-chain mechanism exists to rotate `OldMintController` if the old MC fails or its `Ops` is rotated away.

2. Treasury Movement & Burn Allowance Spending

Affected Components: `governance/src/lib.rs` (`treasury_send`, `burn_from_pool`, `add_hot_wallet`, `remove_hot_wallet`).

Risk: Governance spends against SAC allowances granted by Treasury and Burn Pool. The contract holds no funds and no allowance state; the SAC enforces the spending ceiling. Governance's on-chain controls are limited to OPS auth, `amount > 0`, and the hot-wallet allowlist

on destinations. No per-tx or daily cap exists at the contract level; the effective ceiling is whatever the SAC allowance currently allows.

What we are verifying:

- OPS auth and destination allowlist enforced on every call.
- `treasury_send` and `burn_from_pool` correctly identify Governance as the spender; Treasury and Burn Pool are immutable post-`initialize`.
- Reserved-address rejection for `to` is not enforced (gov-self, treasury, burn-pool, SAC). Any of these in the hot-wallet allowlist creates fund-stuck or allowance-burn surfaces.
- Allowance lifecycle (size, expiration cadence, revocation) is entirely off-chain. A WASM upgrade preserves the live allowance for the successor code.

3. Vault Position Accounting

Affected Components: `vault/src/lib.rs` (`register_deposit`, `draw`, `withdraw`, `emergency_withdraw`, `set_position`, `deposit_cap`, velocity helpers).

Risk: Per-partner / per-asset `Position` is the authoritative bookkeeping for partner liquidity. Position state evolves across four mutation paths (deposit, draw, withdraw, emergency withdraw) and must remain consistent under partner removal, cap changes, and the velocity / ref-replay protections layered on top.

What we are verifying:

- All position mutations use `checked_add` / `checked_sub`; position cannot become negative.
- `register_deposit` and `withdraw` enforce `require_partner_allowed`; `draw` does not (filed as Informational).
- `register_deposit` does not verify a corresponding SAC inflow — OPS-side trust assumption.
- `DepositCap(partner)` is stored per-partner but enforced per-(partner, asset); cross-asset cap bypass is structurally available.
- `withdraw` and `emergency_withdraw` follow CEI ordering (`set_position` before SAC transfer); `draw` does not.
- Replay protection (`UsedRef`) is in persistent storage with explicit TTL extension on write.

4. Rate Oracle & Band Throttle

Affected Components: `vault/src/lib.rs` (`update_rate`, `admin_override_rate`, `compute_band_pct`, `draw` band branches).

Risk: OPS sets exchange rates per (`deposit_asset`, `payment_asset`) pair. A band throttle compares the new rate to the immediately-prior rate (`PrevRate`) and pauses draws at $\geq 15\%$ deviation, throttles at 5–14%. The band check is per-step; a sequence of sub-threshold updates can accumulate arbitrary drift without triggering pause or override. The first `update_rate` on a new pair skips writing `PrevRate`, leaving the throttle dormant for the first draw.

What we are verifying:

- `update_rate` and `draw` correctly, read and apply the band thresholds.
- Rate staleness is enforced on `draw` (36h window).
- `admin_override_rate` is admin-only and toggles `RateBandOverride(pair)`.
- The override flag is not cleared when consumed by `draw`; it persists until the next $\geq 15\%$ rate update overwrites it.
- `update_rate` and `admin_override_rate` are not gated by `require_not_paused`; an override or rate can be staged during a pause and take effect immediately on unpauses.
- Integer truncation in `compute_band_pct` means a 14.999% move rounds to 14 and enters the throttled bucket rather than the paused bucket.

5. Role-Based Access Control & Co-Sign

Affected Components: `mint-controller/src/lib.rs` (`set_admin`, `set_ops`, `set_treasury`), `governance/src/lib.rs` (`set_admin`, `set_ops`), `vault/src/lib.rs` (`set_admin`, `set_ops`).

Risk: Three roles (admin / OPS / partner) with documented separation of duties. Role rotation requires both the current and incoming role-holder to co-sign. No pairwise distinctness check is enforced — admin and OPS can be set to the same address in two consecutive co-signed transactions, collapsing the separation.

What we are verifying:

- Every state-modifying endpoint enforces exactly the documented role check (admin-only, OPS-only, partner-only).

- Co-sign is enforced on `set_admin`, `set_ops`, and `set_treasury` (MC).
- No on-chain check prevents `new_admin == ops` or `new_ops == admin`; role collapse is achievable in two admin-signed transactions.
- `initialize` does not enforce pairwise distinctness of `admin` / `ops` / `treasury` / `burn_pool` / `sac`.
- Old MC's role rotation predates co-sign hardening — its `set_admin`, `set_ops`, `set_treasury` are admin-only, with no timelock or multisig.

6. Pause Coverage & Partner Emergency Exit

Affected Components: `mint-controller/src/lib.rs` (`pause`, `unpause`), `vault/src/lib.rs` (`pause`, `unpause`, `emergency_withdraw`), Governance (no pause primitive).

Risk: Pause coverage is asymmetric across the three contracts. The vault's `emergency_withdraw` is the only on-chain non-Reyts authority path — partners can exit even while the contract is paused and even after `remove_partner`. That guarantee is conditional on the admin not having stripped the partner's wallet allowlist, which is not enforced on-chain.

What we are verifying:

- `mint_to_treasury`, `register_deposit`, `draw`, `withdraw` revert when their contract is paused.
- `emergency_withdraw` does not call `require_not_paused` or `require_partner_allowed` — by design.
- `emergency_withdraw` requires `partner.require_auth()` bound to the specific args (per-invocation Soroban auth), and the destination must be in `PartnerWallet(P, *)`.
- Admin can `remove_partner_wallet(P, *)` for every wallet, blocking emergency exit — no partner-side primitive to self-add wallets.
- Governance has no pause primitive; under OPS compromise, the only on-chain response is a co-signed `set_ops` rotation.
- The MC's SAC admin handoff family (`propose_sac_admin`, `execute_sac_admin_handoff`, `cancel_sac_admin_handoff`) is not gated by `is_paused`.

7. Upgrade, SAC Admin Handoff & Storage TTL

Affected Components: `*/src/lib.rs` (`propose_upgrade`, `execute_upgrade`, `cancel_upgrade`, `propose_sac_admin`, `execute_sac_admin_handoff`), legacy old MC's instance storage on mainnet.

Risk: All three new contracts share a 24h WASM upgrade timelock. SAC admin handoff in Governance and Mint Controller has a 5h timelock — but is inert today because the SAC admin is held by the immutable old MC, which has no transfer function. Old MC's instance storage receives no TTL extension calls in its source and grows monotonically with every mint (UsedRef accumulation in instance storage). The chain-through pattern is structurally permanent per the team's own documentation.

What we are verifying:

- `execute_upgrade` and `execute_sac_admin_handoff` enforce admin auth and the ledger-delay check against the proposal ledger.
- `propose_*` functions write the pending state unconditionally — a second proposal silently overwrites the prior pending hash / address without an event.
- WASM upgrade preserves the contract address and therefore inherits the SAC-side (`treasury`, `governance`) and (`burn_pool`, `governance`) allowances.
- Old MC's instance entry on mainnet shows `liveUntilLedger = 0`; auto-restoration is paid per transaction.
- Old MC's `UsedRef` set is stored in instance (single ledger entry, ~64 KB cap). Currently 21 entries; ceiling estimated at ~800–1,600.
- Persistent storage TTL on `Position` and `UsedRef` in vault and new MC is extended only on write; views call `bump_instance` but do not refresh persistent TTL.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	1	1	1
Medium	2	2	2
Low	5	5	5
Informational	7	7	7
Total	15	15	15

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
H-01	Due to storing all refHashes in the instance storage of the old mint controller, the new mint controller will be permanently DoSd after ~1300-1400 mints	High	Fixed
M-01	Rate band override mechanism can be circumvented with a sequence of transactions	Medium	Fixed
M-02	Deposit cap checks are ineffective as they're per partner, rather than per asset	Medium	Fixed
L-01	Not removing certain instance key-value pairs but only setting them to false can lead to storage bloat	Low	Fixed
L-02	vault.draw does not check partner_allowed	Low	Fixed
L-03	Governance has no per tx/ daily cap on treasury_send and burn_from_pool	Low	Fixed
L-04	All propose_* functions silently overwrite pending state	Low	Fixed
L-05	The treasury address in the Governance contract can't be changed	Low	Fixed

I-01	Use <code>__constructor</code> instead of <code>initialize</code>	Informational	Fixed
I-02	WASM upgrade inherits SAC allowance	Informational	Fixed
I-03	First <code>update_rate</code> on a pair leaves <code>PrevRate = 0</code>	Informational	Fixed
I-04	No check to make sure that <code>throttled_max</code> is less than <code>max_draw_per_tx</code>	Informational	Fixed
I-05	If the contract isn't initialized at 00:00 GMT+0 (Stellar timestamp), the daily caps/limits for the first day will be valid for less than 24 hours	Informational	Fixed
I-06	The new <code>minting_gateway</code> daily cap must remain less than the <code>old_minting_gateway</code> daily cap	Informational	Fixed
I-07	<code>draw</code> does not validate <code>payment_amount</code> against the on-chain rate	Informational	Fixed

High Severity Issues

H-01: Due to storing all refHashes in the instance storage of the old mint controller, the new mint controller will be permanently DoSd after ~1300-1400 mints

Severity: High	Impact: High	Likelihood: Medium
Files: mint-controller/lib.rs	Status: Fixed	

Description:

The current minting mechanism works by initiating the mint from the new mint_controller contract by the OPS role, which has all the necessary checks, checks the refHash, stores it in permanent storage, and then calls the `mint_to_treasury` on the old mint_controller:

```
let old_mc = old_mint_controller(&e);
    let to = treasury(&e);

    // Chain through old MC: old_mc.mint_to_treasury(amount, ref_hash)
    // Old MC is SAC admin and calls SAC.mint() internally.
    // This contract is old MC's OPS, so old MC's require_ops_auth passes.
    let _res: () = e.invoke_contract(
        &old_mc,
        &Symbol::new(&e, "mint_to_treasury"),
        vec! [&e, amount.into_val(&e), ref_hash.into_val(&e)],
    );
```

This is because the old mint_controller is the sac admin on the ReytsCAD asset, while the new mint_controller serves as the OPS role in the old one.

The problem is that unlike the new/current mint_controller, the old one stores the 32 bytes ref hashes in the instance storage (below excerpt is from the old mint_controller contract):

```
fn mark_ref_used(e: &Env, r: &BytesN<32>) {
    e.storage()
        .instance()
```

```
.set(&DataKey::UsedRef(r.clone()), &true);  
}
```

This is problematic as the instance storage is limited to 64 Kb, and after reaching that limit, the contract will no longer be useful and any write/re-writes to instance storage would fail and the transaction would revert.

After roughly ~1300-1400 mints, the instance storage will be filled up and the old mint_controller would revert whenever `mint_to_treasury` is invoked via the new mint_controller. This would lead to a permanent DOS of the minting functionality.

Recommendations:

Re-deploy the ReytsCAD contract and set the new mint_controller as the sac admin to that contract. Discard the old mint_controller contract, and directly mint via the new one, rather than redirecting the call to the old mint_controller (this is because the old mint controller is immutable and non-upgradeable).

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

Medium Severity Issues

M-01: Rate band override mechanism can be circumvented with a sequence of transactions

Severity: Medium	Impact: High	Likelihood: Low
Files: vault/lib.rs	Status: Fixed	

Description:

The RateBandOverride mechanism in `update_rate` exists so that if a rate update results in a difference larger than 15% when compared to the previous rate, to reset the “band” to false. This mechanism is in place so that an admin has to “override” it afterwards, i.e. approving and confirming that the 15%+ increase is legitimate.

If there has been a 15% increase an admin hasn’t overridden it, then `draw` will fail:

```
if band_pct >= 15 {  
    // >=15%: draws paused unless admin override  
    if !get_rate_band_override(&e, &deposit_asset_sac,  
&payment_asset_sac) {  
        panic_with_error!(&e, VaultError::RateBandPaused);  
    }  
    // Overridden – fall through to normal max  
    if deposit_amount > max_draw_per_tx(&e) {  
        panic_with_error!(&e, VaultError::DrawAmountExceedsMax);  
    }  
}
```

The problem is that this mechanism can be circumvented with a sequence of transactions, as there’s no time limit as to how many times the rate can be updated within a day/hour/minute/seconds, and it compares the rate only to the previous one:

```
let old_rate = get_rate(&e, &deposit_asset, &payment_asset).unwrap_or(0);  
let band_pct = compute_band_pct(old_rate, new_rate);
```



```
// Store previous rate before overwriting
if old_rate > 0 {
    e.storage().instance().set(
        &DataKey::PrevRate(deposit_asset.clone(), payment_asset.clone()),
        &old_rate,
    );
}
// Set new rate + timestamp

e.storage().instance().set(
    &DataKey::Rate(deposit_asset.clone(), payment_asset.clone()),
    &new_rate,
);
let ts = e.ledger().timestamp();
e.storage().instance().set(
    &DataKey::RateUpdatedAt(deposit_asset.clone(),
payment_asset.clone()),
    &ts,
);

// Reset band override if >=15% move
if band_pct >= 15 {
    e.storage().instance().set(
        &DataKey::RateBandOverride(deposit_asset.clone(),
payment_asset.clone()),
        &false,
    );
}
```

The OPS role can circumvent this by multiple sequential transactions, here's an example:

- 3 transactions, one after another:

- old rate: 5.1e7
 - update 1: 5.8e7 (13.7%)
- old rate: 5.8e7
 - update 2: 6.2e7 (12%)

- old rate: 6.2e7

- update 3: 7.1e7 (14.5%)

- The total difference between the initial rate and the last update will be 39.2% and the rate band override mechanism was never triggered.

Customer Response:

Fixed in [502d09a](#) and [5c24c26](#).

Fix Review:

Fix Confirmed.

M-02: Deposit cap checks are ineffective as they're per partner, rather than per asset

Severity: Medium	Impact: High	Likelihood: Low
Files: vault/lib.rs	Status: Fixed	

Description:

Whenever a position is being registered, `get_position` is keyed on `(partner, asset_sac)`. The check compares only the current asset's balance against the cap — not the partner's total balance across all deposited assets.

If we make the assumption that in the future there will be more than one `deposit_asset` in the vault. Hypothetically a partner deposits into two different assets (e.g., NGNC and USDC), and each asset position is checked independently against the same single `deposit_cap` value.

This represents an ineffective way of capping assets. For example let's say that the `deposit_cap` for the said partner is set at `50_000e7` units.

- This would mean that the USD equivalent of the NGNC position can be up to ~36 USD.
- While at the same time, the USD equivalent of the USDC position can be up to ~50,000 USD.

This is an ineffective way of capping the positions that a partner can have if more than one deposit asset is involved.

From another perspective, even if we have two of the same-type assets (e.g. USDC and USDT), and a deposit cap of `50_000e7`, this would mean that the partner will be able to hold positions in total of `100_000e7` as the positions are independently checked against the deposit cap, but they're never summed together.

Recommendations:

Enforce a deposit cap which is per partner and per asset.

Customer Response:

Fixed in [502d09a](#).



Fix Review:

Fix Confirmed.

Low Severity Issues

L-01: Not removing certain instance key-value pairs but only setting them to false can lead to storage bloat

Severity: Low	Impact: Medium	Likelihood: Low
Files: vault/lib.rs, governance/lib.rs	Status: Fixed	

Description:

In multiple contracts, there are instance storage key-value pairs which aren't removed but set to "false" during the removal process.

In the Governance, this is the removal of hot wallets:

```
pub fn remove_hot_wallet(e: Env, addr: Address) {
    require_init(&e);
    let a = admin(&e);
    a.require_auth();
    bump_instance(&e);

    set_hot_allowed(&e, &addr, false);

    HotWalletRemoved { addr }.publish(&e);
}
```

Which is performed in the following way:

```
fn set_hot_allowed(e: &Env, a: &Address, allowed: bool) {
    e.storage()
        .instance()
        .set(&DataKey::HotAllow(a.clone()), &allowed);
}
```

And there are similar “removal” procedures in the Vault as well:

- `remove_partner`
- `remove_partner_wallet`
- `remove_hot_wallet`

Considering that these records aren’t removed from the instance storage, depending on how many are added/removed over time, this can lead to storage bloat, as the instance storage is limited to 64KB. Reading/Writing to instance storage may become very expensive over time due to the ever-expanding mapping, or even lead to a possible DOS if the storage reaches 64 KB.

Recommendations:

Considering that some of these key-value pairs aren’t removed for compliance reasons, and depending on the number of hot wallets/partners that are planned to be introduced over time, some/all of these key-value pairs can be transferred to the persistent storage.

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

L-02: vault.draw does not check partner_allowed

Severity: Low	Impact: Low	Likelihood: Low
Files: vault/src/lib.rs	Status: Fixed	

Description:

In `vault/src/lib.rs`, every partner mutating function calls `require_partner_allowed(&e, &partner)` to enforce the documented "removed partners can no longer transact" invariant. The `draw` function does not.

After admin calls `remove_partner(P)`, OPS can still successfully invoke `draw(partner=P, ...)` against P's residual position. The protocol's documented "removal halts further activity for this partner" invariant is not enforced on the `draw` rail.

Recommendations:

Add `require_partner_allowed(&e, &partner)` to `draw` function. If `draw` after `remove_partner` is intentionally a liquidation path for offboarding scenarios, expose it as a separate admin function rather than allowing the OPS to inherit that capability.

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

L-03: Governance has no per tx/ daily cap on `treasury_send` and `burn_from_pool`

Severity: Low	Impact: Low	Likelihood: Low
Files: governance/lib.rs	Status: Fixed	

Description:

`treasury_send` and `burn_from_pool` validate only `amount > 0` and the hot-wallet allowlist. There is no per-tx or daily on-chain cap, the effective per-call ceiling is whatever SAC allowance is currently active. This contrasts with the mint controller and the vault, both of which maintain on-chain rate limits independent of any external allowance grant.

Any path that reaches `treasury_send` or `burn_from_pool` inherits the full current allowance as its blast radius. The protocol's spending discipline therefore depends entirely on the off-chain allowance lifecycle rather than on enforceable on-chain bounds.

Recommendations:

Add per-tx and daily caps inside governance, mirroring the existing pattern in the mint controller and vault.

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

L-04: All `propose_*` functions silently overwrite pending state

Severity: Low	Impact: Low	Likelihood: Low
Files: mint-controller/lib.rs, vault/lib.rs, governance/lib.rs	Status: Partially fixed.	

Description:

Every `propose_*` flow across the three contracts writes the new pending hash / address and the new proposal ledger unconditionally. None of them check whether a prior proposal is already pending, and none emit a `*Replaced` or `*Canceled` event for the implicit cancellation of the previous proposal.

A second call before the original timelock has been executed silently replaces the prior pending state. Off-chain monitors that track state cannot detect the swap, which creates a telemetry gap around the timelock window.

Recommendations:

Either revert in `propose_*` when a proposal is already pending, or emit a `*ProposalReplaced{old, new}` event so off-chain monitors can reconcile the implicit cancellation.`

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

L-05: The treasury address in the Governance contract can't be changed

Severity: Low	Impact: Low	Likelihood: Low
Files: governance/lib.rs	Status: Fixed	

Description:

The mint_controller allows for the Treasury address to be changed by an admin:

```
pub fn set_treasury(e: Env, new_treasury: Address) {
    require_init(&e);
    require_admin_auth(&e);
    new_treasury.require_auth();
    bump_instance(&e);

    let old_treasury = treasury(&e);
    e.storage()
        .instance()
        .set(&DataKey::Treasury, &new_treasury);
}
```

If the treasury address is changed (and this is also plausible on the old mint_controller as well), we will ReytsCAD to the new treasury address. The problem is that there's no way to change the Treasury in the Governance contracts, besides from upgrading the contract itself.

Since the Governance contract has the functionality to move funds from the Treasury address, this would be temporarily impossible until a new wasm upgrade has been executed.

Recommendations:

Include a `set_treasury` function in the Governance contract which would allow for a treasury address change. It should be performed by an admin, and ideally would include a time lock for security purposes.

Customer Response:



Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

Informational Severity Issues

I-01: Use `__constructor` instead of `initialize`

Description:

The mint-controller, governance, and vault contracts all expose an `initialize()` function intended to be called once after deployment. The function only checks `!is_initialized` and `admin.require_auth()` against the caller supplied admin parameter. Between the deploy transaction and the legitimate initialize transaction, any actor can submit `initialize(admin=their_addr, ...)` signed by their own key. Soroban SDK 23 supports the `__constructor` lifecycle hook, which runs atomically with deployment eliminating the race window structurally.

Recommendations:

Replace each `initialize(...)` function with `__constructor(...)`

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

I-02: WASM upgrade inherits SAC allowance

Description:

`execute_upgrade` swaps the contract's bytecode while preserving its address. SAC allowances are stored on the SAC contract keyed by (owner, spender = `governance_address`), so the existing `Treasury` → `Governance` and `BurnPool` → `Governance` allowances carry over to the new code. The replacement bytecode can call `SAC.transfer_from` and `SAC.burn_from` against those allowances without re-authorization from the fund owners and without enforcing the checks present in the previous version.

Recommendations:

Add a mandatory pre-upgrade step to the upgrade runbook: `Treasury` and `BurnPool` sign `SAC.approve(amount=0)` on the `Governance` spender pair before `execute_upgrade` is called. The new wasm then lands with zero spending authority and must be explicitly re-granted post-upgrade.

Customer Response:

Operational runbook procedure changed to revoke allowance prior to upgrades.

Fix Review:

Fix Confirmed.

I-O3: First `update_rate` on a pair leaves `PrevRate` = 0

Description:

The first call to `update_rate` for any new asset pair skips writing `PrevRate`. The write is gated on `if old_rate > 0`, and `old_rate` is 0 on a first-ever call because the underlying storage read defaults to zero. The next `draw` therefore reads `prev_rate = 0`; `compute_band_pct(0, current_rate)` short-circuits to 0; and the band-throttle check falls into the normal-max branch regardless of how off-market the first posted rate is. Band-throttle protection is dormant between the first and second `update_rate` for any new pair.

Recommendations:

Write `PrevRate = new_rate` on the first call so the next `draw` computes against a real baseline:

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

I-04: No check to make sure that `throttled_max` is less than `max_draw_per_tx`

Description:

The `max_draw_per_tx` is a cap that is enforced on draws during normal conditions (rate difference between previous and current rate is 4% or less), while the `throttled_max_draw` cap is enforced whenever the rate is $>4\%$, this serves as a risk parameter to make sure that during abnormal market circumstances, draws are “more limited”.

Although the mechanism functions as intended, there's no checks in the `initialize`, as well as a setter function which makes sure that `throttled_max_draw` is less than `max_draw_per_tx`.

Recommendations:

Introduce checks in `initialize` as well as setter functions which make sure that `throttled_max_draw` is less than `max_draw_per_tx`.

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

I-05: If the contract isn't initialized at 00:00 GMT+0 (Stellar timestamp), the daily caps/limits for the first day will be valid for less than 24 hours

Description:

There are multiple daily limits/caps which are calculated based on the `current_day_index`:

```
fn current_day_index(e: &Env) -> u64 {  
    let ts = e.ledger().timestamp();  
    ts / SECONDS_PER_DAY  
}
```

Considering that the `day_index` is calculated based on the current timestamp (unix timestamp), which is based on Stellar's timestamp.

The problem is that if the contract isn't deployed at/around 00:00 (GMT+0) Stellar timestamp, the duration of the cap will be much shorter the first day, this is because we're dividing the unix timestamp with 86_400.

Recommendations:

Deploy and initialize the contract at/around 00:00 GMT+0.

Customer Response:

Operational runbook procedure update to deploy contract between 23:30 and 00:30 GMT+0.

Fix Review:

Fix Confirmed.

I-06: The new minting_gateway daily cap must remain less than the old_minting_gateway daily cap

Description:

Every successful new MC mint chains through old MC, and both contracts enforce their own independent **DailyCap**. The effective per-day mint ceiling is `min(new_cap, old_cap)`. If a new MC's cap is ever raised above the old MC's, mints will pass the new MC's check but revert to the old MC's cap once **MintedToday** crosses it, silently breaking the pipeline.

Recommendations:

In `set_daily_cap`, reject values larger than old MC's **DailyCap**.

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

I-07: draw does not validate payment_amount against the on-chain rate

Description:

`draw` accepts both `deposit_amount` and `payment_amount` as parameters.

The `deposit_amount` serves as the on-chain position deduction param and for the cap checks (considering that is the amount being withdrawn).

Although `payment_amount` is emitted in the `DrawExecuted` event it is never validated against the on-chain `current_rate`.

Recommendations:

Compute the expected payment amount on-chain ($\text{deposit_amount} * \text{current_rate} / 10^7$) and reject calls where the supplied `payment_amount` diverges by more than an acceptable rounding tolerance.

Customer Response:

Fixed in [502d09a](#).

Fix Review:

Fix Confirmed.

General Recommendations

Through its work with numerous protocols, Certora has built substantial expertise in ongoing monitoring and what sustains protocol security beyond the static guarantees of an audit. Based on this experience, we offer these advisory recommendations to help teams maintain long-term security.

Operational security extends beyond monitoring into the privileged-signing flow itself. Teams should put hardware multisigs and software signers under formal key-management procedures, and rehearse role rotations regularly. The assumptions the contracts rely on — separation of duties across privileged roles, no overlap between privileged addresses and contract allowlists, the signing-tier model — should hold throughout the signing chain, not only at on-chain execution time. A compromise or misconfiguration at the signing layer bypasses every on-chain control built on top of it.

Off-chain complementary audits are equally important. Many of the trust assumptions contracts rely on are enforced exclusively by off-chain code — reconciliation engines, API guards, schedulers, Key Management System infrastructure, and oracle feeds. Smart contracts are only as secure as those components remain. A periodic security review of the off-chain control plane at a cadence aligned with on-chain audits is recommended, and the boundary between the two should be explicitly documented so neither audit assumes the other has covered it.

Below, we expand on each of these areas so teams can apply them consistently and keep the protocol resilient over time.

Findings Conditional on Admin Integrity

The findings below are bound by the assumption that the admin key is operated honestly. Under a compromise, each would escalate in severity, enabling direct asset-loss or bypass of an on-chain safety control.

Direct asset-loss paths:

- I-02 (WASM upgrade inherits SAC allowance) — compromised admin proposes upgrade, waits 24h timelock, then drains both Treasury and Burn Pool allowances through successor bytecode that skips every check.

- L-03 (no per-tx/daily cap on `treasury_send/burn_from_pool`) — admin rotates OPS (or collapses roles, see below) and drains the full active SAC allowance in a single tx with no on-chain ceiling.
- L-04 (`propose_*` methods silently overwrites pending state) — compromised admin can swap a pending legitimate upgrade with a malicious one with no `*Replaced` event, blinding off-chain monitoring during the timelock window.

Safety-mechanism neutralization:

- M-01 (rate band circumventable by sequence) — compounded by `admin_override_rate` being admin-only with no expiry; compromised admin can override the band whenever needed.
- I-04 (no check that `throttled_max < max_draw_per_tx`) — admin can set throttle above the normal max, neutralizing the rate-band throttle.
- I-06 (new MC daily cap vs old MC) — admin can raise the daily cap beyond the chain-through ceiling.

Monitoring Candidates

This section captures recommendations that the protocol team should consider integrating into runtime monitoring.

- `Σ Position(partner, asset) ≤ SAC.balance(vault, asset)` — no on-chain accumulator. Strict inequality (sum > balance) signals fake-credit drift or accounting bugs.
- `SAC.total_supply ≈ Apaylo CAD balance` — within configured tolerance; already monitored off-chain by the reconciliation engine.
- `Governance hot-wallet allowlist ≡ Vault hot-wallet allowlist` (where intentionally shared) — currently independent storage; drift not reconciled.
- `RateBandOverride(pair) == false between rate regimes`. Sticky flag, never automatically cleared after consumption.
- `NGNC Total Supply Increase/Decrease` — Monitoring for sudden abnormal increases/decreases in supply which might point to a problem.
- `RateUpdated event timestamps per asset pair` — Too frequent or stale updates could indicate a problem. Alerting when above/below a certain threshold.

- Upgrade proposal lifecycle (all three contracts) - Alerts on proposal/execution and cancellation events.
- SAC admin rotation proposal (MintController, Governance) - Alerts on proposal/execution and cancellation events.
- Role rotation events (all three contracts) - Notification/event tracking for all role rotations;
- Minted vs SAC.total_supply — alert when an equal increase does not match a Minted event in SAC.total_supply

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.